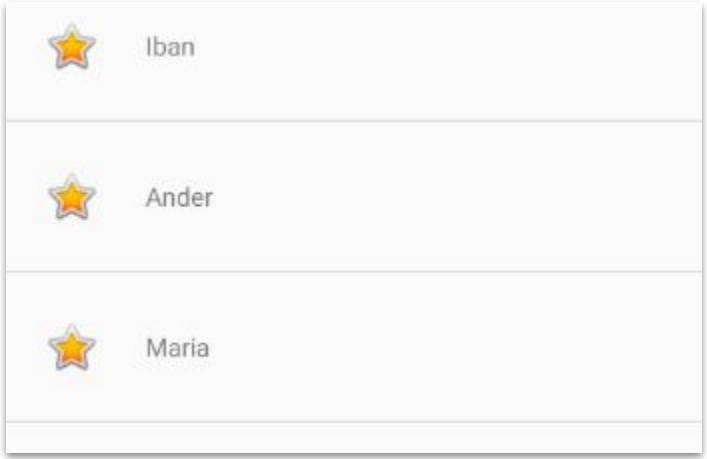
RecyclerView



RecyclerView

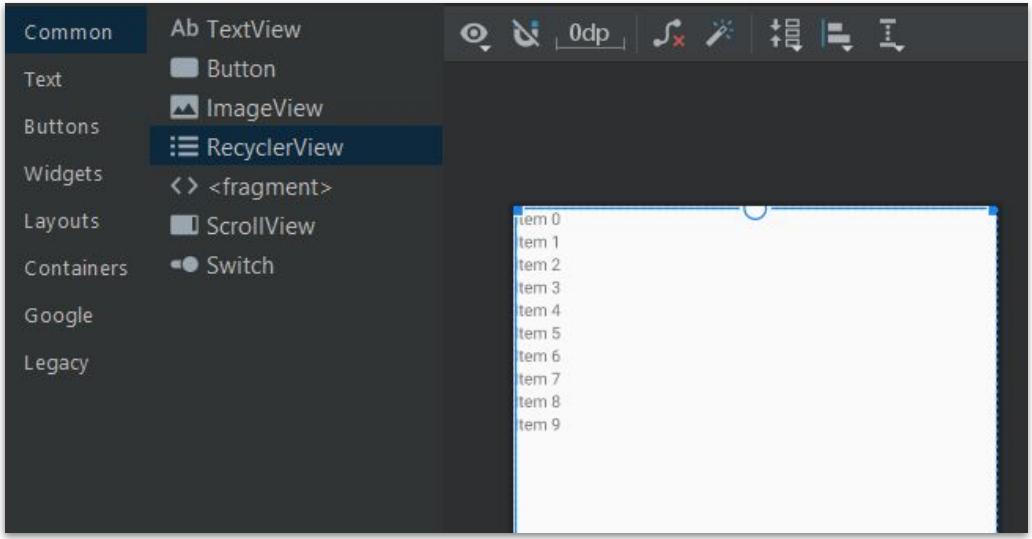
Es un elemento muy usado en Android, la mayoría de aplicaciones hacen uso de él o de las versiones anteriores ListView o GridView que veremos más adelante.

Es realmente una lista con diferentes *Views* pudiendo ser un View un botón un botón con un texto o algo más complejo, lo que queramos.



RecyclerView

Lo primero que vamos a hacer es agregar el *RecyclerView* sobre un contenedor vacío



RecyclerView

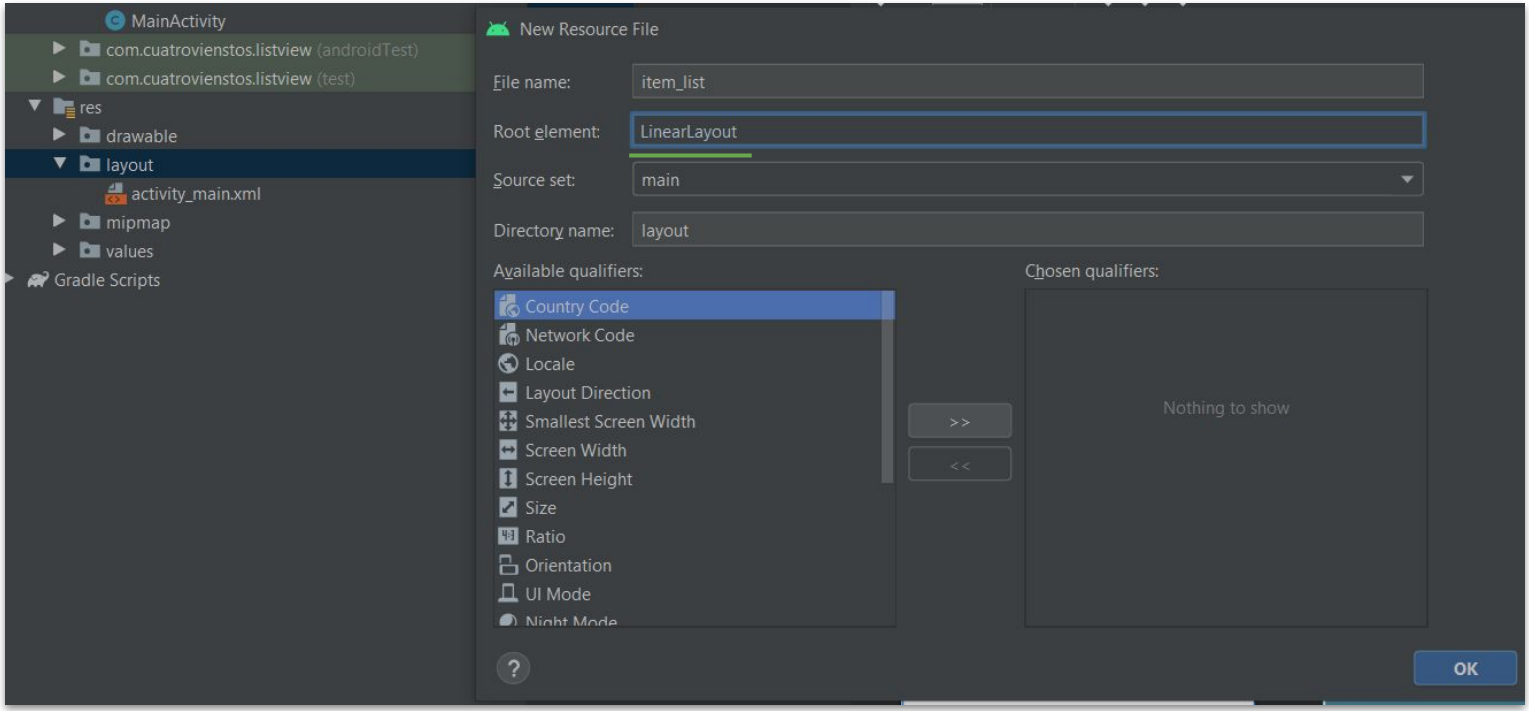
Una vez hecho esto, crearemos un nuevo layout para las *View*, es decir el aspecto que tendrá cada uno de los elementos o líneas que queremos mostrar en forma de lista

En este caso solo se añadirá un texto sobre un *LinearLayout* para probar, pero está View puede ser tan compleja como se quiera.

Sobre la carpeta *layout* hacemos botón derecho New -> Layout Resource File

Recordad marcar en el campo *RootElement* el tipo de layout, en nuestro caso *LinearLayout* que es el elemento más apropiado y a su vez sencilla para que luego se repite uno detrás de otro.

RecyclerView

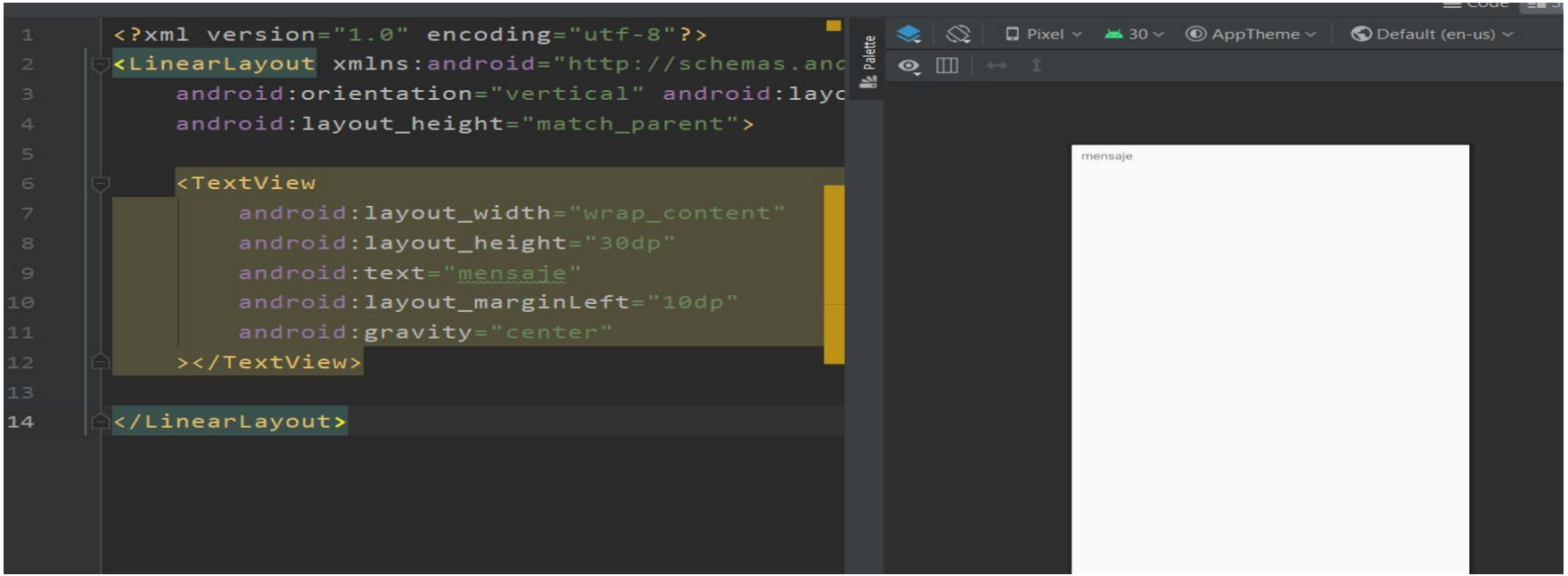


RecyclerView

Como hemos dicho anteriormente en nuestro layout introduciremos únicamente un texto, un *TextView*

Se le especificara una serie de campos para que quede estéticamente más bonito pero poco más

RecyclerView



RecyclerView

Ahora viene la parte más novedosa, para poder relacionar nuestro *RecyclerView* con el *View* o layout que acabamos de crear y los datos, necesitamos una clase adaptador

Esta clase adaptador se va a encargar de recoger una lista con datos y un layout y cargar la información de la lista de cada elemento de la lista utilizando ese layout sobre el *RecyclerView*

Crearemos una clase Java a la altura de MainActivity con el nombre *RecyclerDataAdapter* que deberá heredar de *RecyclerView.Adapter* que necesita de otra clase holder que crearemos dentro de esta misma llamándola *RecyclerDataHolder*

```
public class RecyclerDataAdapter extends RecyclerView.Adapter<RecyclerDataAdapter.RecyclerDataHolder>{}
```

RecyclerView

Utilizando la ayuda que nos proporciona Alt+Enter para resolver los herreros, corregiremos todos ellos seleccionando la primera de las opciones que nos ofrece.

- Primero sobre la clase holder, si pulsamos esta combinación de teclas se nos creará esta clase dentro de *RecyclerDataAdapter*
- El siguiente error nos indica que debemos reescribir ciertos métodos obligatoriamente debido a que heredamos de *RecyclerView.Adapter*
- El siguiente error hace referencia a que nuestro holder debe heredar de *RecyclerView.ViewHolder*
- El último error hace referencia a que falta un constructor sobre nuestra clase *RecyclerDataHolder*

RecyclerView

Nuestro adaptador recibirá una lista desde el MainActivity que utilizará para rellenar los datos.

Para poder trabajar con ella crearemos un atributo en nuestro adaptador del tipo de la lista para poder trabajar con ella y un constructor que reciba esta lista como parámetro

```
ArrayList<String> listData;

public RecyclerDataAdapter(ArrayList<String> listData) {
    this.listData = listData;
}
```

RecyclerView

Algo similar tenemos que hacer en la clase *RecyclerDataHolder* que se encarga de mantener en memoria todos los datos.

En los casos en los que nuestro RecyclerView tenga un scroll y haya datos que no se estén mostrando en pantalla, la forma de proceder de Android es tener renderizados y en memoria sólo los elementos que se muestran en pantalla.

Cada vez que hagamos scroll se volverá a llamar al adaptador que tendrá que hacer un *findViewById*, siendo esta llamada bastante pesada.

Utilizando esta técnica llamada “View Holder Pattern” conseguimos que ésta llamada pesada se realice una vez.

RecyclerView

```
public class RecyclerDataHolder extends RecyclerView.ViewHolder {

    TextView data;

    public RecyclerDataHolder(@NonNull View itemView) {
        super(itemView);
        data = itemView.findViewById(R.id.TextViewData);
    }
}
```

RecyclerView

Ahora debemos dar contenido a los métodos que se han generado

- onCreateViewHolder: Enlaza el adaptador con el item_list.
- onBindViewHolder: Enlaza el adaptador con el holder.
- getItemCount: Devuelve el número de elementos de la lista.

RecyclerView

onCreateViewHolder

```
public RecyclerDataHolder onCreateViewHolder(@NonNull ViewGroup parent, int viewType)
```

En este método tenemos que crear un objeto del tipo *RecyclerDataHolder* para devolverlo.

Según su constructor, para crearlo necesitaremos de una *View*, esta será inflada con el contenido de nuestro layout item_list.

Si el item_list utiliza propiedades que hacen referencia al parent, como match_parent, deberemos pasar como segundo parámetro el objeto parent en lugar de null.

```
public RecyclerDataHolder onCreateViewHolder(@NonNull ViewGroup parent, int viewType) {
    View view = LayoutInflater.from(parent.getContext()).inflate(R.layout.item_list, parent,  attachToRoot: false);
    return new RecyclerDataHolder(view);
}
```

RecyclerView

onBindViewHolder

```
public void onBindViewHolder(@NonNull RecyclerDataHolder holder, int position)
```

En este método enlazamos el contenido de la lista con el *TextView* al que accedemos a través del holder

Para ello vamos a crear un método en el holder que asigne al TextView un String que pasaremos como parámetro

```
public void asignData(String s) {
    data.setText(s);
}
```

RecyclerView

onBindViewHolder

Desde el método lo único que haremos es llamar a esta función que acabamos de crear, hacemos referencia al holder utilizando el parámetro que nos pasan.

```
@Override
public void onBindViewHolder(@NonNull RecyclerDataHolder holder, int position) {
    holder.asignData(listData.get(position));
}
```

RecyclerView

getItemCount

El más sencillo de los tres, simplemente devuelve la cantidad de items que se van a mostrar y lo podemos sacar fácilmente con el tamaño de la lista.

```
@Override
public int getItemCount() {
    return listData.size();
}
```

RecyclerView

Una vez creado el adaptador, la parte complicada está hecha, ahora solo hace falta utilizarlo

En el ***MainActivity*** crearemos dos atributos, una lista que rellenaremos con datos para pasarsela al adaptador y otro atributo para hacer referencia al RecyclerView.

```
ArrayList<String> dataList;
RecyclerView recyclerView;
```

RecyclerView

Seguidamente en la función *onCreate* conectamos el RecyclerView del layout con nuestro objeto y rellenamos nuestra lista con datos

```
recyclerView = (RecyclerView) findViewById(R.id.recyclerId);

for (int i=0;i<50;i++){
    dataList.add("Posición "+i);
}
```

RecyclerView

Ahora lo que haremos es definir cómo queremos que se muestran los elementos dentro del *RecyclerView*, tenemos 3 maneras:

- Horizontal
  ```
  recyclerView.setLayoutManager(new LinearLayoutManager(this, LinearLayoutManager.HORIZONTAL,false));
  ```
- Vertical
  ```
  recyclerView.setLayoutManager(new LinearLayoutManager(this, LinearLayoutManager.VERTICAL,false));
  ```
- Rejilla
  ```
  recyclerView.setLayoutManager(new GridLayoutManager(this,2));
  ```
- Escalonado
  ```
  recyclerView.setLayoutManager(new
  StaggeredGridLayoutManager(3,StaggeredGridLayoutManager.VERTICAL));
  ```

RecyclerView

Por último, creamos el adaptador pasandole la lista de datos y asignamos el adaptador al RecyclerView

```
RecyclerDataAdapter recyclerDataAdapter = new RecyclerDataAdapter(dataList);
recyclerView.setAdapter(recyclerDataAdapter);
```

RecyclerView

Lo último que vamos a ver sobre este tema es cómo agregar la posibilidad de hacer click sobre cada elemento de la lista

Lo primero que necesitamos es crear una interfaz que nos ayudará a manejar los eventos, lo haremos al final de nuestro adaptador

```
public interface OnItemClickListener{
    void onItemClick(String name, int position);
}
```

Definiremos la cabecera de un método que reciba los datos a los que voy a querer acceder desde el MainActivity cuando hagan click

RecyclerView

El segundo paso es declarar un atributo en el adaptador con el tipo de dato de la clase anónima que acabamos de crear y hacer

Además también haremos que el constructor reciba como parámetro este tipo de dato al ser construido y que asigne el parámetro a nuestro atributo de la misma forma que hemos hecho con la lista

```
private ArrayList<String> listData;
private View.OnClickListener listener;
private OnItemClickListener itemListener;


public RecyclerDataAdapter(ArrayList<String> listData, OnItemClickListener listener) {
    this.listData = listData;
    this.itemListener = listener;
}
```

RecyclerView

El tercer punto a modificar es el método assignData de nuestro Holder para que reciba un objeto del tipo de nuestra clase anónima.

En este punto lo que haremos es modificar la cabecera, para que además de recibir los datos que necesitemos para rellenar nuestro objeto Holder reciba también un OnItemClickListener, es decir, un objeto de nuestra clase anónima

```
public void assignData(final String s, final OnItemClickListener onItemClickListener) {
```

RecyclerView

En el cuerpo de este método tendremos que llamar al método setOnClickListener sobre el itemView (se recibe como parámetro en el constructor) que hace referencia al layout completo, lo que sería en nuestro ejemplo el item_list.

Con esto lo que hacemos es reaccionar cada vez que se pulse sobre un elemento de la lista.

En el método onClick que se nos autogenera deberemos llamar al método onItemClick de nuestra clase onItemClickListener pasándole los datos que queremos que lleguen a nuestra MainActivity y que definimos al crear esa clase anónima.

RecyclerView

```
public void assignData(final String s, final OnItemClickListener onItemClickListener) {

    data.setText(s);

    itemView.setOnClickListener(new View.OnClickListener() {
        @Override
        public void onClick(View view) {
            onItemClickListener.onItemClick(s,getAdapterPosition());
        }
    });
}
```

RecyclerView

Como hemos cambiado la función assignData tenemos que cambiar también la llamada a esta función en el método onBindViewHolder.

Con este punto, terminamos con el adaptador

```
@Override
public void onBindViewHolder(@NonNull RecyclerDataHolder holder, int position) {
    holder.assignData(listData.get(position),itemListener);
}
```

RecyclerView

En este punto únicamente nos queda utilizar el adaptador mejorado

El cambio estaría en el constructor únicamente al que le tenemos que pasar un objeto del tipo OnItemClickListener

Para facilitar este proceso escribiremos en el constructor el primer parámetro que será nuestra lista y para el segundo parámetro utilizaremos la ayuda de Android Studio escribiendo new y pulsando Ctrl+Espacio para que nos autogenere el código de igual forma que hacemos cuando queremos programar el click en un botón.

Se nos generará el código que veremos a continuación

RecyclerView

Se ha generado una método onItemClick al que le llegan los datos que hemos definido en el adaptador.

```
recyclerDataAdapter = new RecyclerDataAdapter(dataList, new RecyclerDataAdapter.OnItemClickListener() {
    @Override
    public void onItemClick(String name, int position) {
        Toast.makeText( context: MainActivity.this, text: "Position: "+position,Toast.LENGTH_SHORT).show();
    }
});
```

Ahora con estos datos que recibimos desde este método conocemos sobre qué elemento de la lista se ha hecho click y además todos los datos adicionales que hayamos querido pasar, en nuestro caso un String pero podría haber sido un dato más complejo como un objeto, lo que hemos mencionado al principio de este apartado